

图像处理

1. mt9v03x 采集图像
 - 1.1. 如何使用偏振片
2. 图像预处理（阈值分割）
3. 边线循迹（状态判断、切换）
4. 障碍物识别
 - 4.1. 雷区检测
 - 4.1.1. 相机标定与逆透视变换
 - 4.2. 台阶检测（跳跃障碍物）
 - 4.3. 单边桥检测
 - 4.3.1. 20th: 最长白列识别角点>6
 - 4.3.2. 21th: 相机标定、自适应阈值、角点检测（近处的精度高）
 - 4.4. 大障碍物检测（颠簸路段）
5. 显示屏ips200 测试调试
6. 上位机调试
 - 6.1. WIFI 串口模块
 - 6.2. WIFI SPI（图传）
 - 6.2.1. 基本参数特性
 - 6.2.2. 如何使用
 - 6.2.3. bug-log
 - 6.3. debugger【usb通讯】
7. CYT4bb7 芯片双核通信
 - 7.1. cross_memory_access
 - 7.1.1. 建立通信方式:
 - 7.2. IPC interrupt
 - 7.3. Shared SRAM
 - 7.3.1. DMA
 - 7.3.2. Cache
 - 7.3.3. flash
 - 7.3.4. TCM
 - 7.3.5. SRAM

- 本文针对:

灰度摄像头mt9v03x灰度摄像头（总钻风）的图像处理进行梳理；

对比英飞凌TC624芯片视觉处理全流程和CYT4bb7上的流程

图像算法绝大部分学习SJTU的RT1064图像处理方案

- 本文属于：

机器视觉（MV）范畴，涉及数字图像处理

（嵌入式c语法知识 数据结构与算法）

不包括：计算机视觉（CV）与AI网络模型交叉

（python、machine learning等）

- 核心处理芯片：CMT4bb7（1核）

视觉任务流程：

图像采集->

边线检测+自适应阈值分割处理->

去畸变+逆透视得到俯视图->

处理俯视图计算角点，检测元素 ->

识别障碍物，切换运动状态

1. mt9v03x 采集图像

▼ All.c

Plain Text

```
1 //TC264 逻辑
2 ips200_init();
3 # 屏幕初始化、
4 # 设置方向、颜色、字体
5
6 mt9v03_init();
7 parafalsh_init(); # 参数保存
8 mt9v03X_set_exposure_time(exposure); # 动态调整曝光时间
9 interrupt_global_enable(1); #开启全局中断
```

TC264芯片当中摄像头通信支持UART和SCCB两种方式；

CMT4bb7芯片当中只支持SCCB通信协议；

(serial camera control bus)

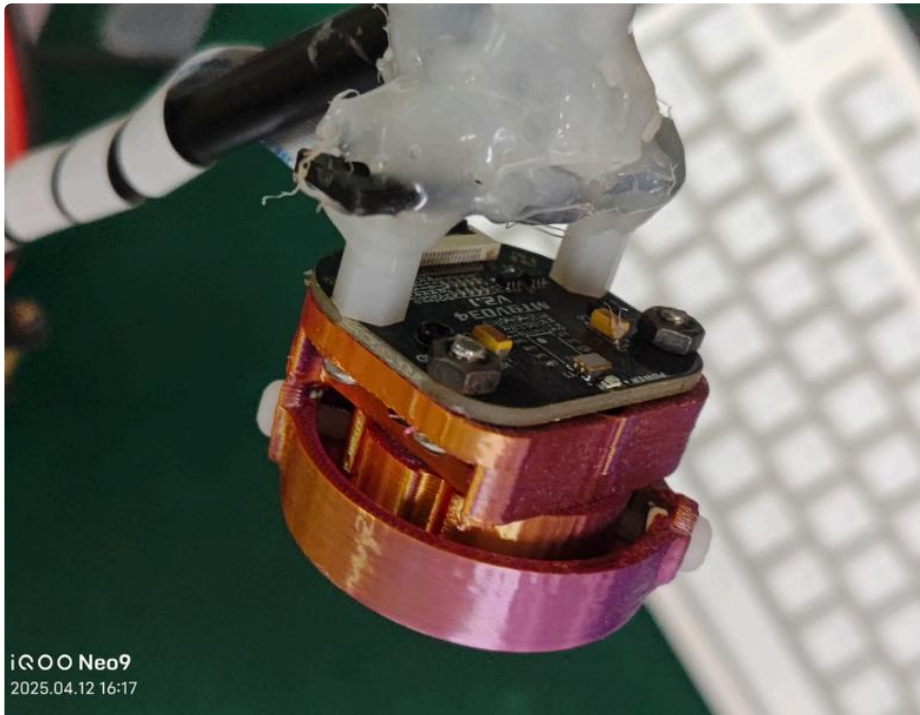
<https://blog.csdn.net/lh66969696/article/details/141498336>

[摄像头参数设置]

1.1. 如何使用偏振片

1. 淘宝网购 物理 实验用的偏振片，然后将文件内三个东西都打印出来（打印机精度要高）；
2. 购买AB胶，M3螺丝（比较短的型号），M3螺母；
3. 将AB胶在偏振片垫片上涂一圈（不要太多），然后按在偏振片上等凝固（注意偏振片要撕掉保护膜）；

4. 用剪刀沿着偏振片垫片裁减一圈
5. 在盖子上内层底部同样涂一层AB胶, 注意不要太多, 然后将偏振片放入盖子内
(上述步骤注意不要把AB胶弄到偏振片中央, 会遮挡镜头)
6. 然后上螺丝装配即可, 拧松盖子上的螺丝可以转动偏振片, 拧紧后可以固定



2. 图像预处理（阈值分割）

▼ Common.c

Plain Text

```

1  //TC264图像处理方案
2
3  # 图像压缩
4  compressimage();
5
6  # 像素处理
7  // Stayguy_ADS(); 图像画线
8  Pixel_filter();
9
10 //摄像头采集: Mt9v03x_image[][]
11 //图像压缩后: Image_Use[][]
12 //阈值分割后: Pixel [][]
13

```

- 头文件定义时限制行、列数，避免溢出，越界访问

▼ segmentation.c

Plain Text

```

1  //CYT4bb采用了局部自适应阈值
2  //block_size = 7;

```

- 灰度处理算法:

1. 用双otsu阈值分割出三个区域，背景黑色，台阶白色，蓝色台阶为灰色；其余颜色阈值在蓝色附近统一处理成灰色
2. 自适应算法+迷宫法（左右手法则）：
 - a. 雷区边线粗细（10cm）++
 - b. 单边桥宽度（45cm），需要搜出边线
 - c. 台阶宽度（50cm）
3. ROI区域、阳光算法（梯度、算子等）：

- 大津法（otsu）：

一种图像分割的自动阈值选择算法，适用于图像灰度分布呈现“双峰”状态图片，采用最大类间方差的原理

$$g = \omega_1 * (\mu - \mu_1)^2 + \omega_2 * (\mu - \mu_2)^2$$

- 活动轮廓模型：

计算量大、内存大。有别于遍历图像像素，该方法更像是几何拓扑的思想进行精细轮廓提取。

属于图像分割的空域技术，通过能量函数计算内、外部能量的原理。[舍！]

【进阶：水平集方法 自动驾驶的语义分割】

3. 边线循迹 (状态判断、切换)

采集到原始图像，先直接进行 (开始main-loop)

1. 边线提取+处理【findedge】

左右手迷宫法+自适应阈值 (计算量小还不用进行手动调阈值)

Lmaze_Adapt ()

Rmaze_Adapt ()

2. 滤波

a. 均值滤波

b. 高斯滤波

c. 三角滤波

3. 采样

4.

findedge ()

{

line

标定---一张映射表完成去畸变+逆透视

滤波---blur

采样

中线跟踪

}

2. 角点提取+筛选【findpoints】

角点检测 (单边桥? 台阶? 障碍路段?)

3. 检查元素【RJTU: 十字、三叉、圆环】

4. 处理中线、近点、远点 (RJTU: 353-450)

a. 没有圆环、十字等元素, 所以不需要前瞻点算斜率角度, 不需要跟踪算法算中线

b. 直接用边线 走直道



```

1  图像 → 边界提取 → 透视变换 → 直接用边线控制
2                                     ↓
3          舵机 = PID(左边界 - 目标距离)
4          台阶 = y坐标判断
5
6  void simple_control(float* left, float* right, int num) {
7      // 1. 直道居中控制：只用边线
8      float left_x = left[5][0]; // 前方第N个点的x
9      float right_x = right[5][0];
10     float center_x = (left_x + right_x) / 2;
11     float error = center_x - target_x; // 目标中点
12
13     舵机 = kp * error;
14
15     // 2. 台阶检测：直接用y坐标
16     for (int i = 0; i < num; i++) {
17         if (is_step(left[i], right[i])) {
18             float dist = left[i][1]; // y = 距离
19             if (dist < JUMP_DIST) trigger_jump();
20         }
21     }
22
23     // 3. 障碍物检测
24     for (int i = 0; i < num; i++) {
25         if (is_obstacle(left[i], right[i])) {
26             float dist = left[i][1];
27             if (dist < BRAKE_DIST) speed = 0;
28         }
29     }
30 }

```

5. 绘制道路元素（线、锚点、角点）

6. 打印main-loop

4. 障碍物识别

4.1. 雷区检测

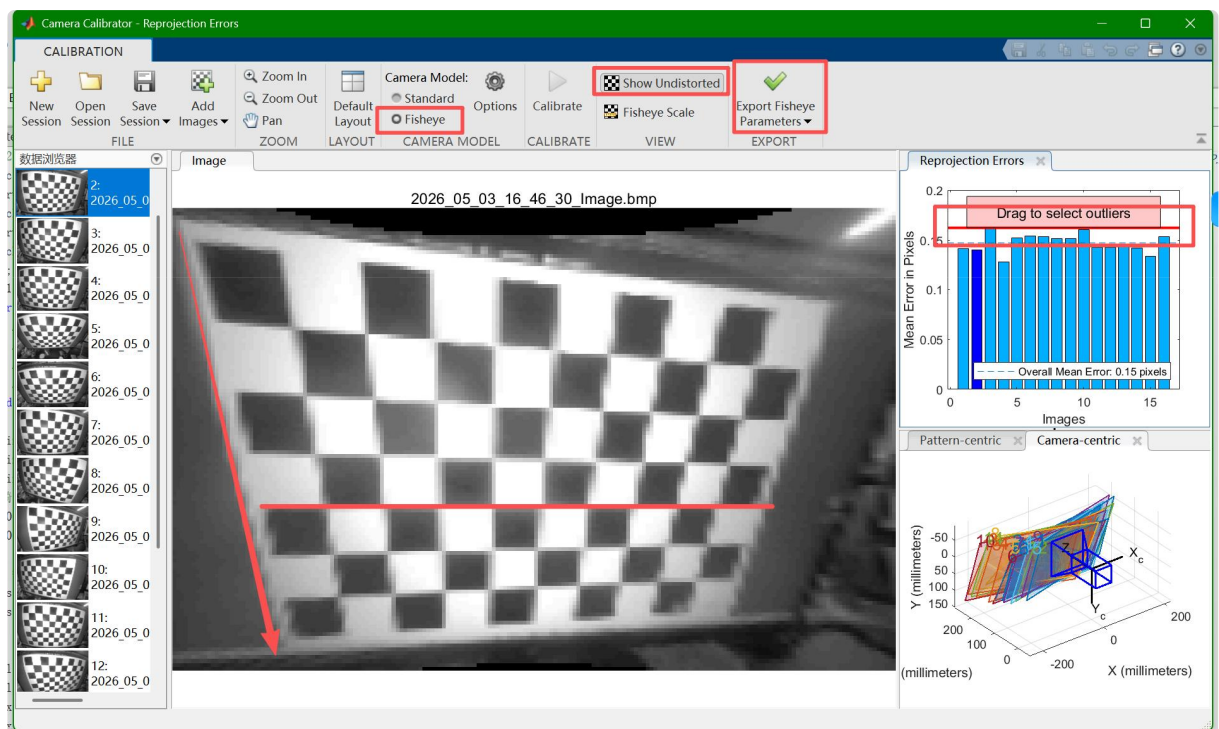
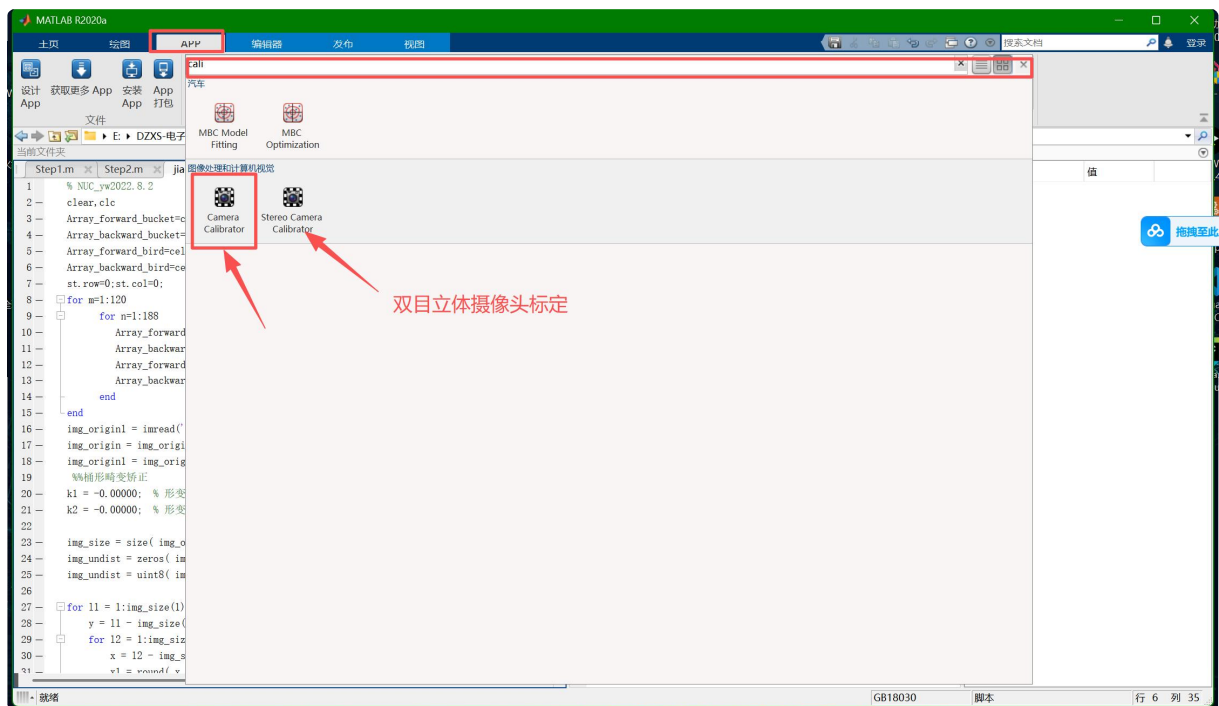
4.1.1. 相机标定与逆透视变换

1. 开源学习一：【SJTU】

教你如何获得摄像头内参矩阵和去畸变参数，以及数学原理

张正友标定法打表（matlab+棋盘格）

- matlab工具箱当中只能导入棋盘格图片，可以直接标定，矫正去畸变，导出畸变参数矩阵



▼ 双目摄像头标定

- **不仅包含内参**：同时计算左右两个摄像头的各自内参。
- **计算外参 (R 和 t)**：确定左摄像头相对于右摄像头的**相对位置** (平移向量 t) 和**相对角度** (旋转矩阵 R)。
- **立体对齐 (Rectification)**：计算如何将两张图校正到同一个平面上，使得同一物体在左右图中的纵坐标 y 完全一致，从而计算**视差 (Disparity)**。
- **输入**：左右两个摄像头**同步拍摄**的成对照片。要求每一组照片中，左右眼都必须能看到同一个标定板。
- **输出**：一个 `stereoParameters` 对象 (包含两个内参对象和一个相对位姿信息)

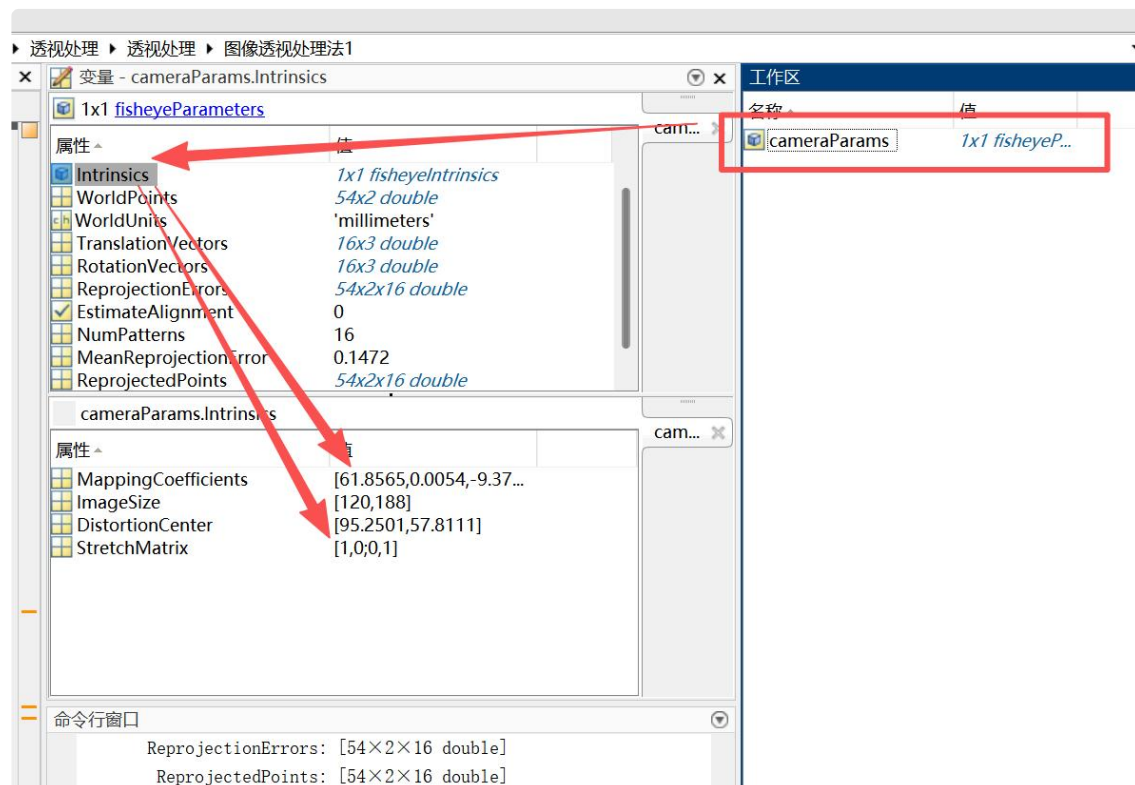
特性	Camera Calibrator	Stereo Camera Calibrator
摄像头数量	1 个	2 个 (固定位置)
主要产出	内参 K 、畸变 D	左右内参 + 相互位置关系 (R, t)
空间概念	仅考虑图像坐标系到相机坐标系	涉及两个相机坐标系之间的转换
典型用途	图像去畸变、单目逆透视 (IPM)	计算深度图 (Depth Map)、三维测量
数学复杂性	较低，只需解单相机的投影方程	较高，涉及对极几何 (Epipolar Geometry)

• STEP 1 :

[链接](#) 【SJTU第三部分--标定与透视变换】

跟随文章教程操作，如上图结果得到去畸变参数和相机内参。

文章内容讲解了逆透视变换数学原理，些许机器人学基础知识和SLAM十四讲当中关于视觉里程计的部分数学原理



输出：

畸变参数--->找Mapping Coefficients

相机内参K--->找DistortionCenter (c_x,c_y) 、 Mapping Coefficients的第一个参数 (焦距fx=fy)

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

- STEP 2:

输出：逆透视变换矩阵（内参和外参的积）

首先鱼眼相机模型去畸变 转换成针孔相机模型。然后四点法计算

2. 开源学习二：【牛爷爷】

拍摄赛道十字照片，选取四个角点进行矫正（手动 or 输入坐标）

教你如何四点法经验计算得到3x3逆透视矩阵，获得物理坐标和像素坐标之间的映射关系

3. 开源学习三：【_苏格拉没有底_】

教你直接获得四张打表变换的映射表，预计算全图像素值，直接查表加速逆变换。同时获得四张映射表之前手动调参矫正畸变

最终输出：

1. 逆透视变换矩阵（3x3）：H和H_inv

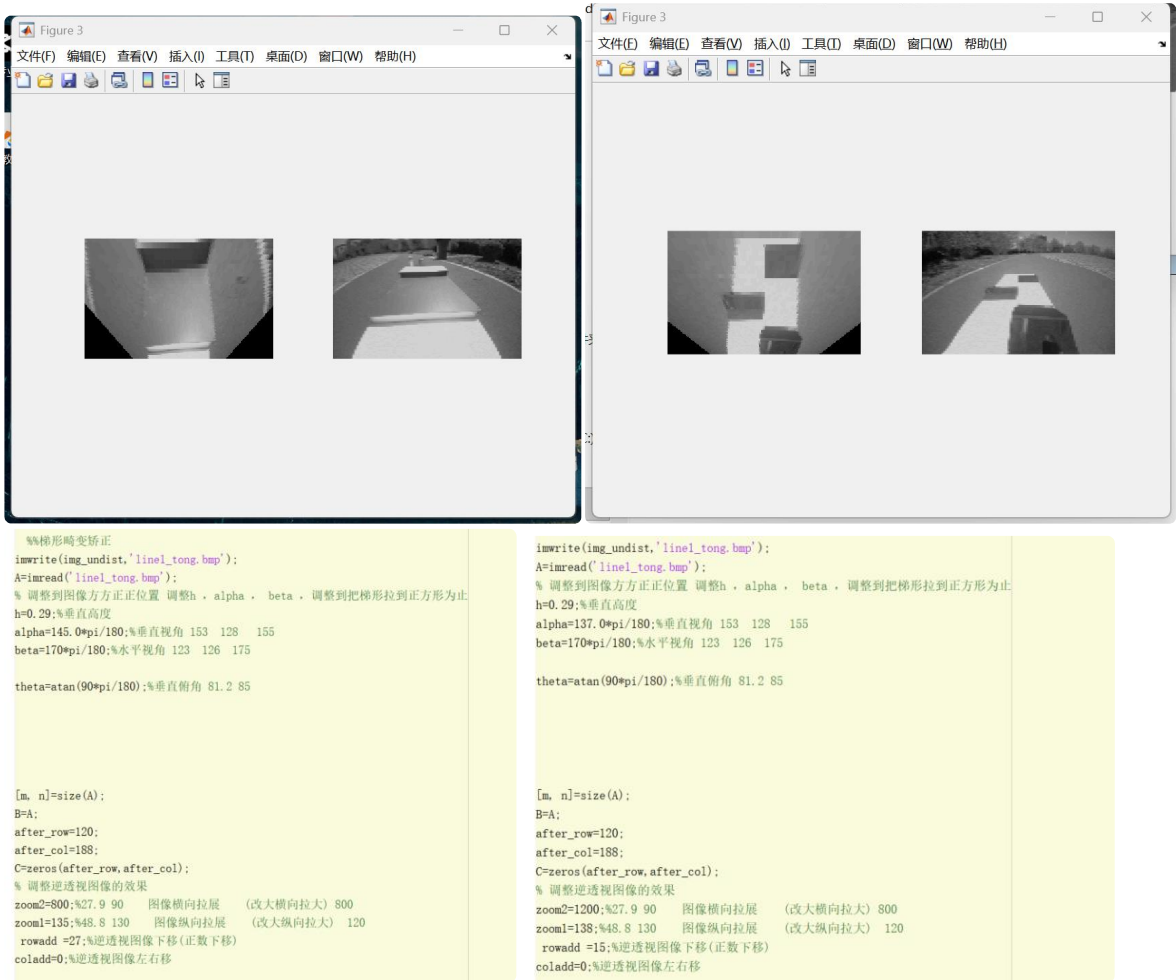
2.mapx、mapy、invx、invy四个映射表

mapx、mapy：原图->标定后俯视图

invx、invy：俯视图->原图

方法一通过matlab当中棋盘格标定输出畸变参数；

方法三则是在matlab当中直接手动水平、垂直矫正；径变、桶型畸变矫正；



现在流程当中的第三步还没验证 只是知道怎么做了，
因为第三步主要就是找一个畸变参数、一个逆透视变换矩阵，一个映射表，
但是这三个东西我找到了三个不同的开源方案，
不知道最后那个矩阵准不准

4.2. 台阶检测（跳跃障碍物）

当台阶的边线（蓝色边线） < 设置好的图像行数

- 1. 正向上台阶
 - a. 抓timing 起跳
 - b. 上蓝色
 - c. 跳出蓝色
- 2. 反向下台阶
 - a. 上坡道识别（边线差值减小 ---变窄）
 - b. 速度不能太快，连下台阶 ----飞坡翻车

4.3. 单边桥检测

【最大疑惑】没看懂

4.3.1. 20th：最长白列识别角点>6

特征一：赛道宽度突变检测（Check_TrackWidth）

核心思想：单边桥赛道宽度从正常 → 突然变宽 → 再突然变窄，形成“近端突变+远端突变”的特征波形。

算法实现

```
// Step 1: 计算每行赛道宽度
for(Ysite=59; Ysite<=OFFLineBoundary+1; Ysite++){
    Road_Width[Ysite] = RightBoundary - LeftBoundary;
}

// Step 2: 检测近端突变（宽度突然增大）
for(Ysite=55; Ysite>=OFFLineBoundary+1; Ysite--){
    if(Road_Width[Ysite] - Road_Width[Ysite-2] >= 5){ // 一阶差分 > 阈值
        Single_BridgeYsite1 = Ysite - 2; // 记录突变行
        break;
    }
}

// Step 3: 检测远端突变（宽度突然减小）
for(Ysite=Single_BridgeYsite1; Ysite>=OFFLineBoundary+1; Ysite--){
    if(Road_Width[Ysite-2] - Road_Width[Ysite] >= 2){
        Single_BridgeYsite2 = Ysite;
        break;
    }
}

// Step 4: 判决条件
if(Single_BridgeYsite1 >= Single_BridgeYsite2 + 1 // 间距至少1行
    && Single_BridgeYsite1 <= 50 // 近端不能太远
    && Single_BridgeYsite2 != 0){ // 远端不能未找到
    TrackWidthChangeFlag = 1;
}
```

宽度变化图示

正常赛道

单边桥赛道

正常宽度

正常宽度

← Road_Width[55~50]

← 近端突变 +5 (Single_BridgeYsite1)

← 宽度突然增大

← 远端突变 -2 (Single_BridgeYsite2)

问题一：赛道宽度的含义

赛道宽度的定义

代码中赛道宽度的计算：

```
Road_Width[Ysite] = ImageDeal[Ysite].RightBoundary - ImageDeal[Ysite].LeftBoundary;
```

即：左边界到右边界的像素距离 = 可行驶的白色区域宽度

单边桥的宽度变化过程

俯视图（从车头往前看）：



图像中近端到远端的宽度变化：

正常：宽度均匀

单边桥：宽度突然增大（近端桥面宽）→ 又突然减小（远端桥面被遮挡消失）

关键点

- 单边桥在近端是“变宽”的，因为桥面是抬起的，视角更近
- 单边桥在远端是“变窄的”，因为桥面延伸到远处时，赛道宽度相对正常赛道变窄
- 整体波形是 宽-窄 的组合，所以代码检测的是宽度突变

```
// 近端：宽度突然增加 (Road_Width[Y] - Road_Width[Y-2] >= 5)  
// 远端：宽度突然减少 (Road_Width[Y-2] - Road_Width[Y] >= 2)
```

特征二：白列台阶检测（Check_Bridge_Left/Right）

核心思想：单边桥导致一边白列突然“升高”，产生不对称的台阶形状。

算法实现（以左单边桥为例）

Step 1 — 构建白列高度数组

```
// 从右往左扫描
for(Xsite = LCDW-20; Xsite > 20; Xsite--){
    for(Ysite = 55; Ysite > 0; Ysite--){
        if(Pixel[Ysite][Xsite] == 0){ // 找到第一个黑点
            White_DOT1[Xsite] = 55 - Ysite; // 白列高度 = 55 - 黑点Y坐标
            break;
        }
    }
    // 记录最长白列的列位置和高度
    if(White_COL_MAX1 <= White_DOT1[Xsite]){
        White_COL_POS1 = Xsite; // 最长白列所在列
        White_COL_MAX1 = White_DOT1[Xsite]; // 最长白列高度
        White_COL_MIN1 = White_DOT1[Xsite - 3]; // 3列后的高度（用于算台阶差）
    }
}
```

Step 2 — 统计台阶左右平坦段

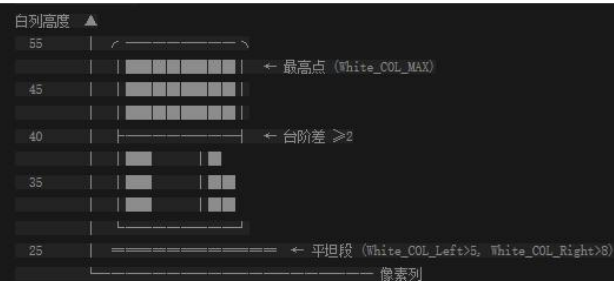
```
// 台阶左侧（靠近桥面）：白列高度差≤1视为平坦
for(Xsite=20; Xsite<White_COL_POS1; Xsite++){
    if(MY_ABS(White_DOT1[White_COL_POS1-1] - White_DOT1[Xsite]) <= 1
    && White_DOT1[Xsite] < 48){
        White_COL_Left1++; // 平坦列计数
    }
}

// 台阶右侧（远离桥面）：同理统计
for(Xsite=LCDW-20; Xsite>White_COL_POS1; Xsite--){
    if(MY_ABS(White_DOT1[Xsite] - White_DOT1[White_COL_POS1]) <= 1
    && White_DOT1[Xsite] < 48){
        White_COL_Right1++;
    }
}
```

白列台阶图示

右单边桥的典型特征（右侧白列突然升高）：





方向: White_COL_Left 方向 (列小) 白列高 → 桥面
White_COL_Right 方向 (列大) 白列低 → 正常赛道

特征三：直线方差不对称 (Straight_Judge)

核心思想：单边桥赛道只有一侧是弯的，另一侧接近直线 → 左右直线方差不对称。

算法实现

```
// Common.c:273-296
float Straight_Judge(uint8 dir, uint8 start, uint8 end){
    // 计算拟合直线斜率 k
    k = (ImageDeal[start].LeftBorder - ImageDeal[end].LeftBorder) / (start - end);

    // 计算每点到拟合直线的方差
    for(i=0; i<end-start; i++){
        Err = (起点直线值 + k*i - 实际值)²;
        Sum += Err;
    }

    return S = Sum / (end - start);
}

// 返回值 S 越小 → 越接近直线
// S <= 1 → 接近直线
// S > 1 → 弯曲
```

左右单边桥的方差特征

单边桥类型	左边界方差 Straight_Left_Varanya	右边界方差 Straight_Right_Varanya
左单边桥	> 1 (弯的)	≤ 1 (直的)
右单边桥	≤ 1 (直的)	> 1 (弯的)
普通弯道	> 1	> 1
直道	≤ 1	≤ 1

问题二：直线方差不对称的理解

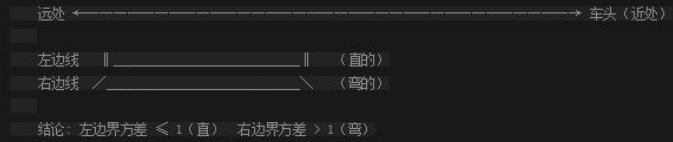
你的理解是正确的。这确实是单边桥的核心特征之一。

左右单边桥的物理含义

左单边桥（左轮上桥，右轮在地面）：



右单边桥（右轮上桥，左轮在地面）：



判决条件对比

特征	左单边桥	右单边桥	普通弯道（两边都弯）
Straight_Left_Varanya	> 1	≤ 1	> 1
Straight_Right_Varanya	≤ 1	> 1	> 1
判决	右<=1 && 左>1	左<=1 && 右>1	两边都>1（被排除）

这不是单边桥区别于普通弯道的特征，单边桥只有 侧向 只有侧向的

特征四：中线方差约束（variance_acc < 20）

```
// Common.c:244-251
void Straightacc_Test(void){
    for(Yaite=55; Yaite>OFFLine+1; Yaite++){
        sum += (Center - 39)^2;
    }
    variance_acc = sum / (54 - OFFLine);
}
```

含义：中线方差小说明赛道方向稳定（接近直道），排除岔路、环岛等复杂路况。

综合判决条件（以左单边桥为例）

```
// Obstacle.c:507-528
if(
    // 特征1: 直线方差不对称
    Straight_Right_Varanya <= 1 // 右直 -> 标志弯在左边
    && Straight_Left_Varanya > 1 // 左弯
    && Straight_Left_Varanya1 > 1 // 远端也弯
    && Straight_Right_Varanya1 > 1

    // 特征2: 白列台阶高度差 ≥ 2
    && White_DOT1[White_COL_POS1] - White_DOT1[White_COL_POS1-3] >= 2

    // 特征3: 平坦段计数（左>8列 右>8列）
    && White_COL_Left1 > 8
    && White_COL_Right1 >= 5

    // 特征4: 赛道宽度突变
    && TrackWidthChangeFlag

    // 特征5: 排除干扰
    && ImageStatus.Left_Line < 3 // 无左边界行
    && ImageStatus.Right_Line < 3 // 无右边界行
    && LeftBoundary_Error < 3 // 左丢线少
    && RightBoundary_Error < 3 // 右丢线少
    && variance_acc < 20 // 中线方差小
    && length_max < 48 // 最大行长度限制
    && Left_Straight_Flag < 5 // 排除已识别为直线的
    && Right_Straight_Flag < 5
){
    Single_Bridge_Left = 1;
    Obstacle_Encoder = 10; // 开始编码器计数
    Buzzer_Count = 6; // 蜂鸣器提示
}
```


问题三：中线方差约束的含义

中线方差的定义

```
// Common.c:244-251
variance_acc = Σ (Center_Y - 39)^2 // 有效行数
```

即：每一行中线到图像中心(列39)的偏差的平方的平均值

物理含义

```
variance_acc 小 (< 20) → 车身方向稳定，行驶在近似直线的道路上
variance_acc 大 (> 20) → 车身方向不断变化，行驶在弯道/复杂路况
```

如何区分单边桥 vs 岔路 vs 环岛

元素类型	中线方差	宽度变化	直线方差	关键区别特征
单边桥	小 (<20)	宽→窄突变	不对称	宽度波形+单侧弯
岔路/十字路口	大 (分叉方向不确定)	突然变宽 (入口宽)	对称或不适用	中心分叉线存在
环岛	大 (绕圈行驶)	近似圆形	对称弯曲	封闭弧形边界
普通弯道	较大 (弯道行驶)	均匀变化	对称>1	无突变波形
直道	最小	稳定均匀	对称≤1	无白列台阶

核心区别逻辑

```
variance_acc < 20 的作用：
├—— 排除岔路：岔路有多条分叉方向 → 中线跳变太
├—— 排除环岛：绕圈行驶方向持续变化 → 中线偏心率大
└—— 保留单边桥：近似直道行驶 → 中线稳定
```

单边桥 vs 普通直道 的区分

这是最关键的问题——单边桥和直道在中线方差上都小，它们的区分靠的是：

判断是否是单边桥：

```
✓ variance_acc < 20      → 排除岔路/环岛 (两者方差都大)
✓ TrackWidthChangeFlag=1 → 排除普通直道 (直道宽度均匀无突变)
✓ 直线方差不对称        → 排除弯道 (弯道两边都弯)
✓ 白列台阶              → 最终确认桥面存在
```

四重条件层层过滤，最终只有单边桥能同时满足所有条件。

4.3.2. 21th:相机标定、自适应阈值、角点检测（近处的精度高

4.4. 大障碍物检测（颠簸路段）

Obstacle.c Plain Text

1

5. 显示屏ips200 测试调试

`cpu0\_main.c`

6. 上位机调试

6.1. WIFI 串口模块

 [逐飞科技 WIFI串口模块说明书 V1.2.pdf](#)

一个数据透传模块，教程内容只有【低速率数据传输】的内容

不适合用来作为摄像头实时图传!!!

1.2.产品特性

- 1) 连接方便:可直接连接到路由器的 WIFI 热点,或者连接电脑释放的热点;
- 2) 传输距离:无障碍、电磁干扰少的环境下,最大传输距离约为 20m,复杂室内环境、电磁干扰严重的情况下,传输距离将会受限;
- 3) 转发速度快:本模块采用高性能单片机(160MHz 主频)来进行数据转发,转发速度更快,模块最大传输速度能够达到 300KB/S 以上(无电磁干扰的情况);
- 4) 支持自定义配置参数:可以采用 AT 指令对模块进行参数配置,实现更多的通信需求;

- 传输速度: 300KB/s -> 带宽只有2.4Mbps

工业或竞赛中常见的“图传”通常传输的是原始像素数据或压缩率较低的图片。如果不经
过高效压缩,即便只是 160 × 120 的彩色图像(约 57KB/帧),以 20fps 传输也需要
约 9 Mbps 的带宽,远超该模块的 2.4 Mbps 上限。

6.2. WIFI SPI (图传)

前备知识: 计网TCP、单片机 DMA

TCP、UDP是计算机网络当中概念, OSI模型当中传输层中进行通信的两种协议。排查错误的时候, 理解哪一层通信出现问题有帮助

6.2.1. 基本参数特性

配合逐飞高速WIFI SPI无线模块, 实现高帧率图传。该模块采用高性能单片机(160MHZ主频) 来进行数据转发, 模块最大传输速度能达到2500KB/S以上(无线环境良好时)。



1.2.产品特性

- 1. 连接方便:可直接连接到路由器的 Wi-Fi 热点,或者连接电脑释放的热点;
- 2. 传输距离:无障碍、电磁干扰少的环境下,最大传输距离约为 10m,复杂室内环境、电磁干扰严重的情况下,传输距离将会受限;
- 3. 转发速度快:本模块采用高性能单片机(160MHz 主频)来进行数据转发,模块最大传输速度可达 3.6MByte/S 以上(无线环境良好时);
- 4. 支持 TCP Client 与 UDP 两种 Socket 通信方式, TCP Server 暂不支持;
- 5. TCP 与 UDP 模式下支持指定本机端口号;
- 6. 支持双向数据传输;
- 7. 支持使用逐飞助手上位机,实现图传、虚拟示波器、参数调试等功能;

- WIFI SPI模块1.0, 最大传输速率达2500KB/s
- WIFI SPI模块2.0 版本, 最大传输速率达3600KB/s

6.2.2. 如何使用

 [逐飞科技 高速WIFI6B21-SPI V2.0模块说明书.pdf](#)

1. 硬件模块：

首先看好接口；

其次知道应该是什么现象，什么意思；

模块上总共有三个灯，分别是红色电源、绿色接收、蓝色发送三个状态灯：

- 1) 红色电源灯：模块供电工作时该红色 LED 亮起，此 LED 如果熄灭证明模块工作异常，应当立即断电检查；
- 2) 绿色接收灯：当收到数据时该 LED 会闪烁，持续接受数据时看起来就像是常亮；
- 3) 蓝色发送灯：当模块发送数据该 LED 会闪烁，持续发送大量数据时看起来是常亮。



高速 WIFI SPI 模块指示灯示意图

2. 软件代码：

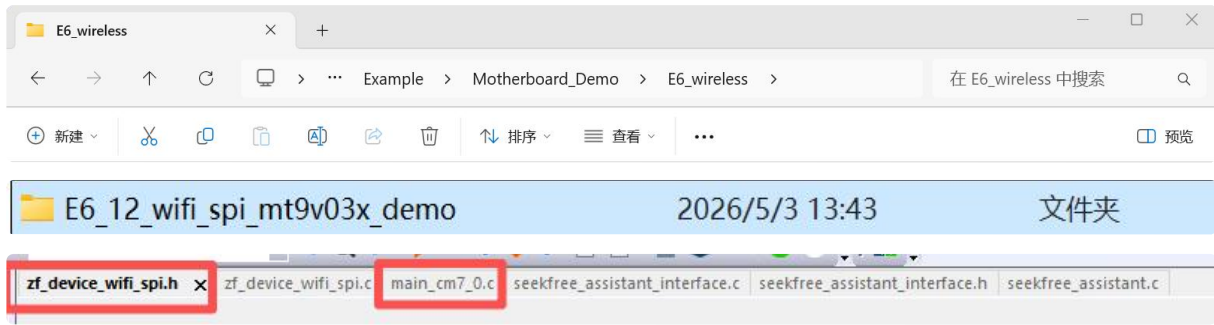
【逐飞助手（上位机调试查看）+逐飞仓库对应芯片例程（下位机需要烧录的代码）】

a. 逐飞助手



（先用图传开始侦听，然后查看串口助手，看看返回的是报错信息还是一堆火星文，接收计数上千的不明文字一般是正常图像编码的传输，各种button详见说明文档。）

b. 图传demo



上述两个文件当中需要更改ip地址、端口、wifi名称密码等等。

注意`main\_cm7\_0.c`当中要取消注释：

```
#if(0 == INCLUDE_BOUNDARY_TYPE)
// 发送总帧图像信息(仅包含原始图像信息)
seekfree_assistant_camera_information_config(SEEKFREE_ASSISTANT_MT9V03X, image_copy[0], MT9V03X_W, MT9V03X_H);
#endif
```

▼ practice

```
1  /*使用seekfree assistant 的时候调用例程就好，最好还是通看一下例程代码，
2  很多注释备注是干什么用的，怎么写的错误检测*/
3  # define WIFI_SSID
4  # define WIFI_PASS
5  # define
```

6.2.3. bug-log

【4.1///按照逐飞助手功能和wifispi例程尝试】

bug1:

- 图像传输没有图片
- 串口助手没有数据发送
- 更改热点名称无中文字符 仍然没有找到问题所在

可能使用的wifi spi不是最新的2.0版本，例程与模块不对应，spi的参数没有调节

当时计网还没学到网络层和传输层，tcp、udp协议不懂

【5.3 ///】

debug1:

例程代码当中没有取消注释，使用串口助手可以看到一开始报错wifi not connected（不要勾选HEX显示，重复出现一堆十六进制数组），发现是ip地址不对应，

之后报错assert `seekfree\_assistant.c`所在路径三百多行有错，发现是没有指明摄像头类别型号，在`main\_cm7\_0.c`当中取消注释，说明camera_information_config就好

时隔一个月二次尝试才问ai问出来，还是耐心不够加畏难情绪，非要万事俱备，知识面储备足够才动手尝试。虽然考虑到时间进度不够多次尝试，但这个bug还是优先级较高，影响推进才解决的。

9.6. 为什么上位机显示这么慢

因为串口传输速度有限，因此上位机刷新的慢。建议购买店铺的 TFT 、IPS114、

IPS200等屏幕，可以直接配套例程在液晶屏上显示图像，比上位机快的多。

6.3. debugger 【usb通讯】

1. assert 断言

【本项目的应用在于不变式检查，确保图像处理的计算稳定性和后续图像算法调试】

```
void assert (int expression) ;
```

- expression 是一个条件表达式。当程序运行到`assert`函数时，会首先计算 `expression` 的值。如果 `expression` 的值为真（非零），则程序继续正常执行；如果 `expression` 的值为假（0），则 `assert`函数会在 标准 错误流中输出一条错误消息，并调用`abort`函数终止程序的执行。
-
- 在调试阶段，`assert`函数可以帮助我们快速定位问题所在；在发布版本中，可以通过定义 `NDEBUG` 宏 来禁用`assert`函数，从而避免运行时开销。
-
- 调试技巧：

1. 参数验证

Plain Text

```
1 void process_buffer(char* buf, size_t len) {
2     // 基础检查
3     assert(buf != NULL);
4     assert(len > 0);
5
6     // 进阶检查：缓冲区对齐
7     assert((uintptr_t)buf % 8 == 0 && "Buffer should be 8-byte aligned");
8
9     // 业务逻辑检查
10    assert(len <= MAX_BUFFER_SIZE);
11
12    //保证内存安全
13    //与业务约束
14    //性能优化检查内存对齐情况
```

▼ 2.状态机与流程控制验证

Plain Text

```
1 void update_player_state(Player* player) {
2     assert(player != NULL);
3
4     switch(player->state) {
5         case IDLE:
6             // 处理闲置逻辑
7             break;
8         case MOVING:
9             // 处理移动逻辑
10            assert(player->velocity > 0); // 移动状态速度应为正
11            break;
12        case ATTACKING:
13            // 处理攻击逻辑
14            assert(player->weapon != NULL); // 攻击必须持有武器
15            break;
16        default:
17            assert(0 && "Invalid player state"); // 捕获未处理的状态
18    }
19 }
20
21 //验证状态转换的合理性
```

▼ 3.不变式检查!!!

Plain Text

```
1 //目的:
2 //1 .确保数据结构的完整性
3 //2. 捕获内存管理错误
4 //3. 验证算法实现的正确性
5
6 struct Vector {
7     size_t capacity;
8     size_t size;
9     int* data;
10 };
11
12 void vector_push_back(Vector* vec, int value) {
13     // 前置条件检查
14     assert(vec != NULL);
15     assert(vec->size <= vec->capacity);
16     assert(vec->capacity == 0 || vec->data != NULL);
17
18     // 扩容逻辑...
19
20     // 后置条件检查
21     assert(vec->size > 0);
22     assert(vec->data[vec->size-1] == value);
23     assert(vec->size <= vec->capacity);
24 }
```

▼ 4.调试版本的性能检查

Plain Text

```

1 void render_scene(Scene* scene) {
2     assert(scene != NULL);
3
4     #ifdef DEBUG
5         // 只在调试版本执行的性能检查
6         clock_t start = clock();
7     #endif
8
9     // 实际渲染逻辑...
10
11    #ifdef DEBUG
12        clock_t end = clock();
13        double elapsed = (double)(end - start)/CLOCKS_PER_SEC;
14        assert(elapsed < 0.016 && "Frame rendering took too long");
15    #endif
16 }
17 //该技巧适合实时系统上的应用

```

▼ 5.自定义断言宏

Plain Text

```

1 #define ASSERT_RANGE(value, min, max) \
2     ASSERT_MSG((value) >= (min) && (value) <= (max), \
3         "Value " #value " out of range [" #min ", " #max "]")
4
5 // 使用示例
6 void set_volume(int volume) {
7     ASSERT_RANGE(volume, 0, 100);
8     // ...

```

7. CYT4bb7 芯片双核通信

7.1. cross_memory_access

共享内存处，0核1核都可以访问。【理解为图像缓冲区】

PS：摄像头和屏幕初始化都是对硬件状态（芯片外设寄存器）直接设置，不用双核都进行初始化或者通信，0核内完成一次配置即可；

7.1.1. 建立通信方式：

1. main文件当中在主函数之前进行定义和地址的声明

```
main_cm7_0:
```

```
a.#define DATA_LENGTH xx
```

(宏定义)

```
b.#pragma location = 0x2808'070c
```

(具体地址编译后查找.map文件)

```
c. __no_init float m7_1_data[DATA_LENGTH];
```

```
main_cm7_1:
```

```
a. #define DATA_LENGTH xx
```

```
b. #pragma location = 0x2808'070c
```

```
c. float m7_1_data[DATA_LENGTH] = {0,0.1,0.2,0.3};
```

2. while循环当中更新Cache

```
main_cm7_0:
```

```
a. system_delay_ms(100);
```

```
b. SCB_CleanInvalidateDCache_by_Addr(&m7_1_data, sizeof(m7_1_data));
```

更新cache

```
c. printf();
```

将数据打印到串口

```
main_cm7_1:
```

```
a. system_delay_ms(300);
```

```
b. SCB_CleanInvalidateDCache_by_Addr(&m7_1_data, sizeof(m7_1_data));
```

7.2. IPC interrupt

结构体封装图片数组

进行传输:

(transmit state)

标志位1: 进入单边桥状态

标志位2: 进入障碍物状态

标志位3: 进入台阶跳跃状态

标志位4: 进入雷区转圈状态

(transmit feature)

雷区 ---> 传输中心坐标

台阶 ---> 传输台阶边缘在2D图片中的位置, 测算距离

(台阶边缘到达图片下方, 传输指令)

单边桥 ---> ?

7.3. Shared SRAM

- 【rt1064---TCM】



```
if (defined(__ICCARM__))
//定义将代码放在ITCM的方式
#define AT_ITCM_SECTION_INIT(var) var @"ITCM_NonCacheable.init"
//定义将代码或者变量放在DTCM的方式
#if ((!(defined(FSL_FEATURE_HAS_NO_NONCACHEABLE_SECTION)) && FSL_FEATURE_HAS_NO_NONCACHEABLE_SECTION))
#define AT_DTCM_SECTION(var) var @"NonCacheable"
#define AT_DTCM_SECTION_ALIGN(var, alignbytes) SDK_PRAGMA(data_alignment = 1)
#define AT_DTCM_SECTION_INIT(var) var @"NonCacheable.init"
#define AT_DTCM_SECTION_ALIGN_INIT(var, alignbytes) SDK_PRAGMA(data_alignment = 1)
#endif
//定义将代码或者变量放在OCRAM的方式
#define AT_OCRAM_SECTION(var) var @"OCRAM_CACHE"
#define AT_OCRAM_SECTION_ALIGN(var, alignbytes) SDK_PRAGMA(data_alignment = 1)
#define AT_OCRAM_SECTION_INIT(var) var @"OCRAM_CACHE.init"
#define AT_OCRAM_SECTION_ALIGN_INIT(var, alignbytes) SDK_PRAGMA(data_alignment = 1)
//定义将代码或者变量放在SDRAM的方式
#define AT_SDRAM_SECTION(var) var @"SDRAM_CACHE"
#define AT_SDRAM_SECTION_ALIGN(var, alignbytes) SDK_PRAGMA(data_alignment = 1)
#define AT_SDRAM_SECTION_INIT(var) var @"SDRAM_CACHE.init"
#define AT_SDRAM_SECTION_ALIGN_INIT(var, alignbytes) SDK_PRAGMA(data_alignment = 1)
//定义将代码或者变量放在SDRAM_NONCACHE的方式
#define AT_SDRAM_NONCACHE_SECTION(var) var @"SDRAM_NonCacheable"
#define AT_SDRAM_NONCACHE_SECTION_ALIGN(var, alignbytes) SDK_PRAGMA(data_alignment = 1)
#define AT_SDRAM_NONCACHE_SECTION_INIT(var) var @"SDRAM_NonCacheable.init"
#define AT_SDRAM_NONCACHE_SECTION_ALIGN_INIT(var, alignbytes) SDK_PRAGMA(data_alignment = 1)
```

【CYT4bb7-----SRAM】

libraries > sdk > common > src > drivers > syslib > C cy_syslib.h > ..

```
#if defined (__ICCARM__)
typedef union { cy_israddress __fun; void * __ptr; } cy_intvec_elem;
/** \cond INTERNAL */
/** Type of ISR callbacks for backward compatibility with the UDB components. Do not use it in your code. */
typedef union { cy_israddress __fun; void * __ptr; } intvec_elem;
/** \endcond */
#endif /* defined (__ICCARM__) */
```

- 受限于芯片内存设置，将图像计算存放在ITCM当中似乎行不通。

CYT4BB7 链接文件 linker_directives_tviibh.icf 支持的内存区域

从 linker_directives_tviibh.icf:205-214:

区域名	地址范围	说明
SELF_ITCM	0x0000_0000 (16KB)	CM7 自映射ITCM
SELF_DTCM	0x2000_0000 (16KB)	CM7 自映射DTCM
ITCM	0xA000_0000 (16KB)	CM7_0 ITCM
DTCM	0xA001_0000 (16KB)	CM7_0 DTCM
SRAM	0x2800_0000	片上SRAM (768KB)
CODE_FLASH	0x1000_0000	代码Flash
WORK_FLASH	0x1400_0000	工作Flash

注意：CYT4BB7 没有 OCRAM_CACHE、SDRAM_CACHE、SDRAM_NonCacheable 这些区域——这些是 RT1064/IMX 系列专有的外部存储区域。
CYT4BB7 是纯片内 SRAM，没有外部 SDRAM。

与 RT1064 的主要差异

对比项	RT1064 (MIMXRT)	CYT4887 (Traveo II)
ITCM/DTCM 大小	各自 128KB	各自仅 16KB
外部 SDRAM	支持 (EMMC/XECC)	不支持
OCRAM 缓存区	有 (含缓存属性)	不支持
AT_SDRAM_SECTION	可用	需删除或空定义

核心建议: CYT4887 的 TCM (ITCM/DTCM) 非常小 (各 16KB), 不适合像 RT1064 那样把大量代码放入。需要改写 `segmentation.c` 中的用法 —— 建议仅将 `clone_image`、`getOSThreshold` 等高频小函数放入 ITCM, 大数组 (如 `Image_Use`、`Pixel`) 放入普通 SRAM 即可。

需要我帮你直接修改 `cy_syslib.h` 添加这些宏吗?

7.3.1. DMA

Direct Memoery Access, 通过硬件实现的数据传输机制。

1. DMA请求
2. DMA初始化
3. DMA启动, 总线出借
4. DMA数据
5. 总线归还
6. DMA停止

大量数据传输, 要求使用DMA

7.3.2. Cache

集成在CPU里的 极高速、容量小 的缓存

- 存在一致性问题
- 地址映射, 没有独立地址
- 存在 “Cache Hit” “Cache Miss”

7.3.3. flash

- 速度慢、容量大的非易失性存储器 (掉电不掉参)
- 需要等待周期 (wait states), 限制CPU性能发挥
- 存放Code和RO-data(只读常量数据)

7.3.4. TCM

【紧耦合存储器】

- ITCM

instruction !!!

运行中断函数、高频算法，使CPU运行不受flash限制

- DTCM

Data! ! ! ! !

存放堆栈Stack、关键变量的数据缓冲区，可极速响应

【CYT4bb7】

TCM 的存在：在 Traveo II 的某些配置中，CM7 核确实有紧耦合内存（TCM），但你会发现它可能被直接映射在特定的地址段（比如 `0x0000_0000` 附近）。

7.3.5. SRAM

【静态随机存取存储器】----易失性存储器 掉电数据丢失

- SRAM常常被分为多块（如 AXI SRAM, SRAM1, SRAM2 等），分布在不同的总线域上，方便 DMA 和 CPU 并发访问而不产生冲突。
- 存放变量、缓存、DMA传输数据

启动控制：CM0+ 是主核（Master），它负责启动 CM7。在 CM0+ 上跑通算法，意味着你已经搞定了基础的系统时钟、Flash 读取和基本的 SRAM 配置。

硬件权限：CM0+ 通常拥有最高的系统访问权限，调试外设（如摄像头接口或显示接口）更直接。

架构差异：CM0+ 是冯·诺依曼架构，没有 CM7 那样的 L1 Cache、FPU（浮点运算单元）和 SIMD（指令级并行）。

- 注意：如果你的图像算法涉及大量 `float` 运算，CM0+ 只能通过软件模拟，速度会极慢。

Traveo II 系列非常强调 **MPU（内存保护单元）**。

- 在 CM0+ 上跑通的代码，搬到 CM7 后可能因为 MPU 没配置导致 **HardFault**。
- **调试建议：**在 CM0+ 验证阶段，尽量保持内存分配在“公共区域”（Shared SRAM），这样两个核都能看到，方便对比调试。